Transformer based compression research project toolkit

This is a notebook with many numerical examples of concepts found in *Elements of Information Theory*

These functions may be useful in future experiments exploring universal source coding and transformer based coding. For now, they are used as an educational tool to reinforce concepts in the book.

In [6]:
```
import numpy as np
import matplotlib.pyplot as plt
from math import comb
from itertools import combinations
```

In [14]:
```
# Concepts: basic entropy calculations

x = [0, 0, 1, 1, 1, 2]

def convert_sample_to_pmf(sample):
    values, counts = np.unique(sample, return_counts=True)
    probs = counts/counts.sum()
    return dict(zip(values, probs))

def clean_pmf(pmf):
    return {k: v for k, v in pmf.items() if v > 0.0}

def generate_bin_pmf():
    p = np.random.rand()
    dict_out = {
        0: p,
        1: 1-p
    }
    return dict_out

def iid_generate(p_dict, N):
    p0 = p_dict[0]
    p1 = p_dict[1]
    return np.random.choice([0,1], size=N, p=[p0, p1])

def exact_binary_sample(p, N):
    count0 = int(round(p[0] * N))
    count1 = N - count0

    samples = np.array([0]*count0 + [1]*count1)
    np.random.shuffle(samples)

    return samples

def entropy_calc(pmf):
```

```
    pmf = clean_pmf(pmf)
    pmf_array = np.asarray(list(pmf.values()))
    return -np.sum(pmf_array * np.log2(pmf_array))
```

In [5]:
```
# Concept: grouping values together means that entropy cannot increase

X = [1, 2, 3, 4, 5, 6, 7, 8]
X_pmf = convert_sample_to_pmf(X)

def coarsening_func(pmf_dict):
    pmf_fdict = {}
    for k, v in pmf_dict.items():
        if k in [1]:
            new_key = 'A'
        else:
            new_key = 'B'
        pmf_fdict[new_key] = pmf_fdict.get(new_key, 0) + v
    return pmf_fdict

print(f'H(f(X)) = {entropy_calc(coarsening_func(X_pmf))} is less than H(X) =
```

```
H(f(X)) = 0.5435644431995964 is less than H(X) = 3.0
```

In [16]:
```
# Concepts: Conditional probability, chain rule

p_joint = {
    (0,0): 0.1,
    (0,1): 0.2,
    (0,2): 0.1,
    (1,0): 0.05,
    (1,1): 0.3,
    (1,2): 0.25
}

def p_X_marg(p_joint):
    p_X = {}
    for (x, y), p in p_joint.items():
        p_X[x] = p_X.get(x, 0) + p
    return p_X

def p_Y_marg(p_joint):
    p_Y = {}
    for (x, y), p in p_joint.items():
        p_Y[y] = p_Y.get(y, 0) + p
    return p_Y

def p_X_given_Y_convert(p_joint):
    p_Y = p_Y_marg(p_joint)
    p_X_given_Y = {}
    for (x, y), p in p_joint.items():
        cond = p / p_Y[y]
        p_X_given_Y[(x, y)] = cond
    return p_X_given_Y

def p_Y_given_X_convert(p_joint):
    p_X = p_X_marg(p_joint)
```

```
    p_Y_given_X = {}
    for (x, y), p in p_joint.items():
        cond = p / p_X[x]
        p_Y_given_X[(x, y)] = cond
    return p_Y_given_X

def cond_entropy_Y_given_X(p_joint):
    h_Y_X = 0
    p_X = p_X_marg(p_joint)
    for x in p_X:
        cond_pmf = {}
        for (x2, y), p in p_joint.items():
            if x2 == x:
                cond_pmf[y] = p / p_X[x]
        slice_entropy = entropy_calc(cond_pmf)
        h_Y_X += p_X[x] * slice_entropy
    return h_Y_X

def cond_entropy_X_given_Y(p_joint):
    h_X_Y = 0
    p_Y = p_Y_marg(p_joint)
    for y in p_Y:
        cond_pmf = {}
        for (x, y2), p in p_joint.items():
            if y2 == y:
                cond_pmf[x] = p / p_Y[y]
        slice_entropy = entropy_calc(cond_pmf)
        h_X_Y += p_Y[y] * slice_entropy
    return h_X_Y

# implementing chain rule checks
# verify H(X,Y) = H(X|Y) + H(Y)

H_XY = entropy_calc(p_joint)
H_Y_given_X = cond_entropy_Y_given_X(p_joint)
H_X = entropy_calc(p_X_marg(p_joint))

print(H_XY, H_Y_given_X + H_X)
print(f'error: {H_XY - (H_Y_given_X + H_X)}')
```

```
2.365957320949175 2.3659573209491747
error: 4.440892098500626e-16
```

Jensen's inequality + concavity

The following code implements a concavity check of the entropy calc function for the binary case.

In [17]:
```
# implement something like f(lambda * x1 + (1-lambda)*x2) <= lambda * f(x1)

def numerical_jansen_check(p, q, func):
    lams = np.linspace(0, 1, 50)
    results = []
    for lam in lams:
```

```
        m = {k: lam*p[k] + (1-lam)*q[k] for k in p.keys()}
        lhs = func(m)
        rhs = lam*func(p) + (1-lam)*func(q)
        results.append(lhs >= rhs)
    return all(results)

pmf1 = generate_bin_pmf()
pmf2 = generate_bin_pmf()

# Verify the concavity of entropy function
entropy_concave_check = numerical_jansen_check(pmf1, pmf2, entropy_calc)
print(entropy_concave_check)
```

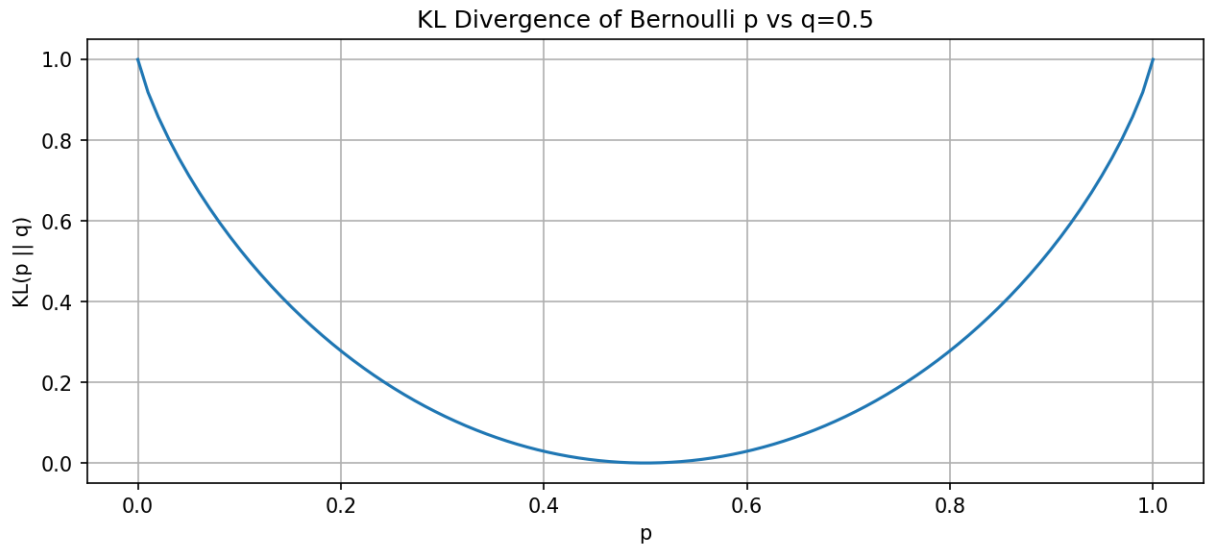
```
True
```

Kullback-Leibler divergence

In the following python exercises we develop functions to calculate the KL divergence of two binary distributions

In [18]:

```
def kl_div(p, q):
    p_val = np.asarray(list(p.values()))
    q_val = np.asarray(list(q.values()))
    mask = p_val > 0
    p_val = p_val[mask]
    q_val = q_val[mask]
    result = np.sum(p_val * np.log2(p_val/q_val))
    return result

probability = np.linspace(0, 1, 100)


kl_values = []

for prob in probability:
    p = {0: prob, 1:1-prob}
    q = {0: 0.5, 1:0.5}
    kl_values.append(kl_div(p, q))

plt.figure(figsize=(10, 4), dpi=150)
plt.plot(probability, kl_values)
plt.xlabel("p")
plt.ylabel("KL(p || q)")
plt.title("KL Divergence of Bernoulli p vs q=0.5")
plt.grid(True)
plt.show()
```



Shannon average code length calculation for realizable codes

When code lengths are expected to be integers which are an unescapable reality in real compression schemes, the word lengths must be rounded up to the nearest integer.

Therefore, we calculate $L = \sum_i p_i \, l_i$ using $l_i = \lceil \log_2 \frac{1}{p_i} \rceil$

In [19]:
```
def shannon_average_code_length_calc_integer(pmf):
    output = 0
    for digit in pmf.keys():
        output += pmf[digit] * np.ceil(-np.log2(pmf[digit]))
    return output

p = generate_bin_pmf()
L = shannon_average_code_length_calc_integer(p)
H = entropy_calc(p)

print(f'probability: {p}')
print(f'Shannon_average_code_length_integer: {L}')
print(f'Entropy: {H}')
print(f'Test for if H < L < H+1: {H < L < H + 1}')
```

```
probability: {0: 0.06671521396162539, 1: 0.9332847860383746}
Shannon_average_code_length_integer: 1.2001456418848762
Entropy: 0.3535441444869463
Test for if H < L < H+1: True
```

Kraft inequality

Below we implement a Kraft inequality verification function

We are implementing the following inequality:

$\sum_{x \in \mathcal{X}} D^{-l(x)} \leq 1$

In [10]:
```
def kraft_inequality_verify(pmf):
    pmf = np.asarray(list(pmf.values()))
    return np.sum(2 ** -(np.ceil(-np.log2(pmf)))) <= 1
```

Universal source coding

We start to get into the meat of what we're really here for: getting average code lengths using what we know about universal source coding!

The Csiszar-Korner universal code length calculation comes from the Elements of Information theory book in chapter 13 (eq. 13.32):

$\ell(x^n) \leq \log_2(n+1) + \log_2 \binom{n}{k} + 2$

In [20]:
```
seq_test = exact_binary_sample(generate_bin_pmf(), 1000)

pmf_find = convert_sample_to_pmf(seq_test)

def csiszar_korner_universal_code_length(seq):
    n = len(seq)
    k = np.sum(seq)
    ltype = np.ceil(np.log2(n+1))
    if k == 0 or k == n:
        lseq = 0
    else:
        lseq = np.ceil(np.log2(comb(n, k)))
    return ltype + lseq

def expected_ck(seq, n):
    total = 0
    blocks = 0

    for i in range(0, len(seq), n):
        block = seq[i:i+n].tolist()
        if len(block) < n:
            block += [0]*(n - len(block))
        b_len = csiszar_korner_universal_code_length(block)
        total += b_len
        blocks += n

    return total / blocks
```

Encoding algorithm

The universal source coding calculation above only calculates the code length bit size

We will implement a type based universal source compressor to more concretely and empirically match our theoretical length calculation

In [21]:
```
def int_to_bits(n, length):
    return [int(b) for b in bin(n)[2:].zfill(length)]

def lex_rank(seq):
    rank = 0
    ones_left = sum(seq)
    k = sum(seq)
    n = len(seq)
    bit_length = int(np.ceil(np.log2(comb(n, k))))

    if ones_left == 0:
        return int_to_bits(rank, bit_length)

    for i, bit in enumerate(seq):
        if bit == 1:
            rank += comb(n - i - 1, ones_left)
            ones_left -= 1
    return int_to_bits(rank, bit_length)


def type_encode(block):
    n = len(block)
    k = sum(block)

    bit_length_k = np.ceil(np.log2(n + 1))
    k_bits = int_to_bits(int(k), int(bit_length_k))
    return k_bits

def full_encode(full_seq, n):
    blocks = []
    full_array = []

    for i in range(0, len(full_seq), n):
        block = full_seq[i:i+n].tolist()
        if len(block) < n:
            block += [0]*(n - len(block))
        blocks.append(block)

    for block in blocks:
        typ = type_encode(block)
        idx = lex_rank(block)

        full_array.append(typ)
        full_array.append(idx)

    flat_stream = [bit for bits in full_array for bit in list(bits)]
```

```
    return flat_stream


block_size = 30
compress_seq = iid_generate(generate_bin_pmf(), 10000)
generated_p = convert_sample_to_pmf(compress_seq)
p = generated_p[1]
bps_empirical = len(full_encode(compress_seq, block_size))/len(compress_seq)
bps_shanon_real = shannon_average_code_length_calc_integer(generated_p)
bps_ex_ck = expected_ck(compress_seq, block_size)
bps_lower_limit = entropy_calc(generated_p)

print(f'Calculating with p = {p}')
print(f'bps_empirical: {bps_empirical}')
print(f'bps_ex_ck {bps_ex_ck}')
print(f'bps_shanon_real {bps_shanon_real}')
print(f'bps_lower_limit {bps_lower_limit}')
```

```
Calculating with p = 0.9695
bps_empirical: 0.3225
bps_ex_ck 0.31007984031936126
bps_shanon_real 1.1525
bps_lower_limit 0.1968931689468837
```

Encoding Algorithm 2

We develop another algorithm for universal source coding. This algorithm will simply sequentially calculate the bit length sequentially

In [22]:
```
def sequential_universal_source_coding(seq, p_array = None):
    total_bits = 0
    k = 0
    i = 0

    for bit in seq:
        if p_array is not None:
            p1 = p_array[i]
        else:
            p1 = (k+1) / (i+2)
        p0 = 1-p1

        if bit == 1:
            total_bits += -np.log2(p1)
        else:
            total_bits += -np.log2(p0)
        k += bit
        i += 1

    bits_per_symbol = total_bits / len(seq)
    return bits_per_symbol


bps_sequential = sequential_universal_source_coding(compress_seq)
bps_lower_limit = entropy_calc(generated_p)
```

```
print(f'Calculating with p = {p}')
print(f'bps_sequential: {bps_sequential}')
print(f'bps_ck_empirical: {bps_empirical}')
print(f'bps_ck_expected {bps_ex_ck}')
print(f'bps_lower_limit {bps_lower_limit}')


half_log_n = 1/2 * np.log2(len(compress_seq)) / len(compress_seq)
two_stage_cost = bps_empirical - bps_sequential

print(f'(1/2)*logn = {half_log_n} actual two stage compression cost: {two_st
```

```
Calculating with p = 0.9695
bps_sequential: 0.1976789414370937
bps_ck_empirical: 0.3225
bps_ck_expected 0.31007984031936126
bps_lower_limit 0.1968931689468837
(1/2)*logn = 0.0006643856189774724 actual two stage compression cost: 0.1248
2105856290632
```

N sweep

This figure examines the relationship between bitlength and size of the sequence

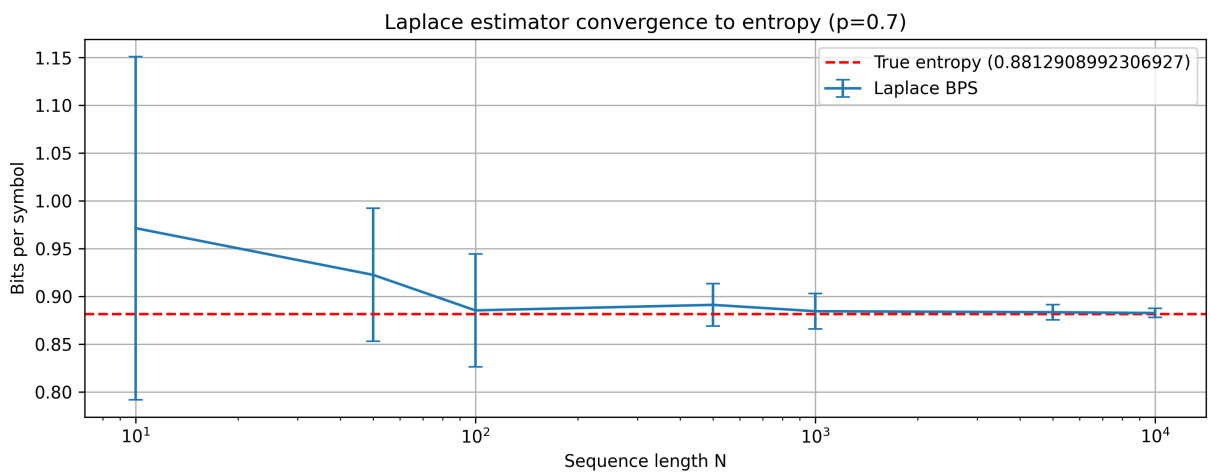
Before doing that let me ask:

- How should we perform a sweep when dealing with probabilities
- I want to change N or sequence length and experiment with how that changes compression rates. Each experiment will have different results though, so I need guidance on how to run probabalistic experiments

In [14]:
```
def bps_vs_N_sweep(p_true, N_values, n_trials=100):
    results = {}
    for N in N_values:
        trial_bps = []
        for _ in range(n_trials):
            seq = iid_generate({0: p_true, 1: 1-p_true}, N)
            trial_bps.append(sequential_universal_source_coding(seq))
        results[N] = (np.mean(trial_bps), np.std(trial_bps))
    return results

N_values = [10, 50, 100, 500, 1000, 5000, 10000]
p_true = 0.7
results = bps_vs_N_sweep(p_true, N_values, n_trials=100)

# unpack properly
means = [results[N][0] for N in N_values]
stds = [results[N][1] for N in N_values]
true_entropy = entropy_calc({0: p_true, 1: 1-p_true})

print(f"True entropy: {true_entropy}")
for N, m, s in zip(N_values, means, stds):
```

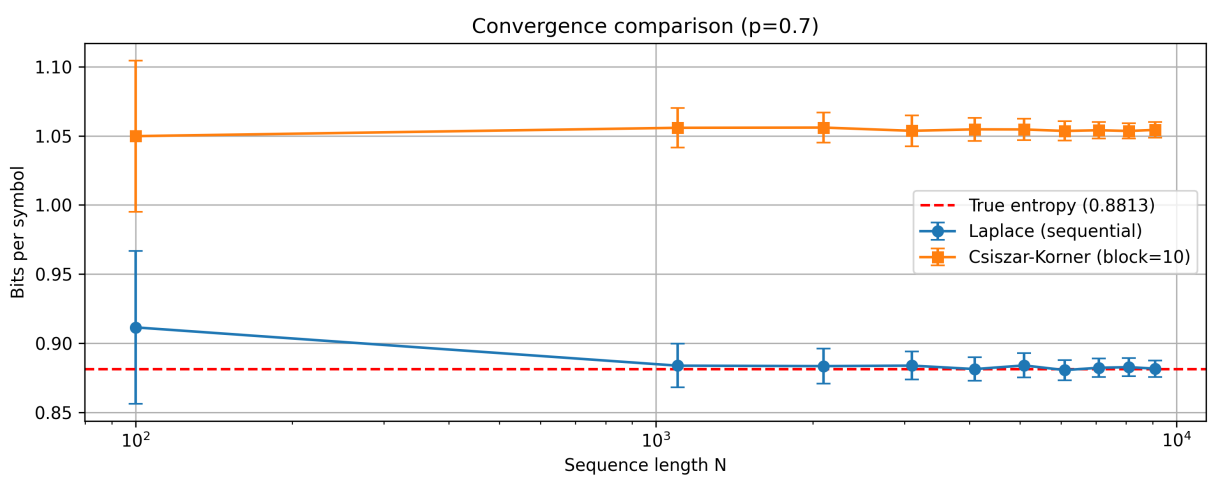
```
    print(f"N={N}  mean={m}  std={s}  gap={m-true_entropy}")

# plot
plt.figure(figsize=(10, 4), dpi=300)
plt.errorbar(N_values, means, yerr=stds, capsize=4, label='Laplace BPS')
plt.axhline(y=true_entropy, color='red', linestyle='--', label=f'True entrop
plt.xscale('log')
plt.xlabel('Sequence length N')
plt.ylabel('Bits per symbol')
plt.title(f'Laplace estimator convergence to entropy (p={p_true})')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
True entropy: 0.8812908992306927
N=10  mean=0.9713788793025938  std=0.1794746106952506  gap=0.090087980071901
11
N=50  mean=0.9225117980413822  std=0.06964662184907462  gap=0.04122089881068
9466
N=100  mean=0.8853679175948806  std=0.05911356861460408  gap=0.0040770183641
87945
N=500  mean=0.8911562404988551  std=0.02210403229097913  gap=0.0098653412681
62409
N=1000  mean=0.8845035514133401  std=0.018323901772891935  gap=0.00321265218
26474352
N=5000  mean=0.8834548794763427  std=0.007795807627726121  gap=0.00216398024
565001
N=10000  mean=0.8826501043424005  std=0.004764019480898613  gap=0.0013592051
117078396
```



In [15]:
```
def bps_vs_N_sweep_ck(p_true, N_values, block_size, n_trials=100):
    results = {}
    for N in N_values:
        trial_bps = []
        for _ in range(n_trials):
            seq = list(iid_generate({0: p_true, 1: 1-p_true}, N))
            trial_bps.append(expected_ck(np.array(seq), block_size))
        results[N] = (np.mean(trial_bps), np.std(trial_bps))
    return results

# run both sweeps
```

```
#N_values = [100, 500, 1000, 5000, 10000]
N_values = np.arange(100, 10000, 1000)
p_true = 0.7
block_size = 10
true_entropy = entropy_calc({0: p_true, 1: 1-p_true})

results_laplace = bps_vs_N_sweep(p_true, N_values, n_trials=100)
results_ck = bps_vs_N_sweep_ck(p_true, N_values, block_size, n_trials=100)

means_laplace = [results_laplace[N][0] for N in N_values]
stds_laplace = [results_laplace[N][1] for N in N_values]
means_ck = [results_ck[N][0] for N in N_values]
stds_ck = [results_ck[N][1] for N in N_values]

plt.figure(figsize=(10, 4), dpi=300)
plt.errorbar(N_values, means_laplace, yerr=stds_laplace, fmt='o-', capsize=4
plt.errorbar(N_values, means_ck, yerr=stds_ck, fmt='s-', capsize=4, label=f'
plt.axhline(y=true_entropy, color='red', linestyle='--', label=f'True entrop
plt.xscale('log')
plt.xlabel('Sequence length N')
plt.ylabel('Bits per symbol')
plt.title(f'Convergence comparison (p={p_true})')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



minGPT

Here we start getting into minGPT. We will perform basic setup and run a few experiments.

We formulate a hypothesis: Laplace estimation for binary IID data approaches entropy as it sees more of a sequence. We hypothesize that this laplace estimation can be done offline by training the transformer to estimate "true p" that a laplace estimation scheme would produce. This offline learning can therefore produce gains for small n's, even for IID data that has no "structure to exploit"

In [ ]:
```python
# We generate our dataset
import torch
from torch.utils.data import Dataset
from torch.utils.data.dataloader import DataLoader
from mingpt.utils import set_seed

class IID(Dataset):
    """
    IID binary sequences
    """

    def __init__(self, n=10, num_samples=10000):
        self.n = n
        self.num_samples = num_samples

    def _sample(self):
        # sample probability p from uniform (0, 1)
        p = torch.rand(1).item()
        # generate IID Bernoulli sequence of length self.n
        seq = torch.bernoulli(torch.full((self.n,), p)).long()

        x = seq[:-1].clone()
        y = seq[1:].clone()

        return x, y

    def get_block_size(self):
        return self.n - 1

    def __getitem__(self, idx):
        return self._sample()

    def __len__(self):
        return self.num_samples

dataset = IID(n=100, num_samples=10000)
```

In [17]:
```python
from mingpt.model import GPT

model_config = GPT.get_default_config()
model_config.model_type = 'gpt-nano'
model_config.vocab_size = 2
model_config.block_size = dataset.get_block_size()
model = GPT(model_config)
```

```
number of parameters: 0.09M
```

In [18]:
```python
# create a Trainer object
from mingpt.trainer import Trainer

device = 'mps'
model = model.to(device)
```

```
train_config = Trainer.get_default_config()
train_config.learning_rate = 1e-4 # the model we're using is so small that w
train_config.max_iters = 100000
train_config.num_workers = 0
trainer = Trainer(train_config, model, dataset)
```

```
running on device cpu
```

In [19]:

```
def batch_end_callback(trainer):
    if trainer.iter_num % 100 == 0:
        print(f"iter_dt {trainer.iter_num}; iter {trainer.iter_num}: train l
trainer.set_callback('on_batch_end', batch_end_callback)

#trainer.run()
#torch.save(model.state_dict(), 'iid_model.pt')

model.load_state_dict(torch.load('iid_model.pt'))
```

Out[19]:

```
<All keys matched successfully>
```

In [20]:

```
model.eval()
```

Out[20]:

```
GPT(
  (transformer): ModuleDict(
    (wte): Embedding(2, 48)
    (wpe): Embedding(99, 48)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-2): 3 x Block(
        (ln_1): LayerNorm((48,), eps=1e-05, elementwise_affine=True)
        (attn): CausalSelfAttention(
          (c_attn): Linear(in_features=48, out_features=144, bias=True)
          (c_proj): Linear(in_features=48, out_features=48, bias=True)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((48,), eps=1e-05, elementwise_affine=True)
        (mlp): ModuleDict(
          (c_fc): Linear(in_features=48, out_features=192, bias=True)
          (c_proj): Linear(in_features=192, out_features=48, bias=True)
          (act): NewGELU()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((48,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=48, out_features=2, bias=False)
)
```

Producing sequences

In this section we will develop our bit generation process and compute the bits/symbol metrics to compare compression schemes

In [21]:
```
import torch.nn.functional as F
from mingpt.model import GPT

model_config = GPT.get_default_config()
model_config.model_type = 'gpt-nano'
model_config.vocab_size = 2
model_config.block_size = 99

model = GPT(model_config)
model.load_state_dict(torch.load('iid_model.pt', map_location='cpu'))
model = model.to('mps')
model.eval()

# generate an array of probabilities given a bit sequence

def transformer_p_array(sequence, model, device='mps'):
    model.eval()
    sequence=torch.tensor(sequence, dtype=torch.long, device=device).unsquee

    with torch.no_grad():
        logits, _ = model(sequence)  # [1, T, vocab_size]
        # convert to probabilities
        probs = F.softmax(logits, dim=-1)  # [1, T, vocab_size]
        # extract probability of token=1 at each timestep
        p_array = probs[0, :, 1].tolist()  # list of floats

    return p_array
```

```
number of parameters: 0.09M
```

In [22]:
```
compress_seq = iid_generate(generate_bin_pmf(), 99)  # 99 because model bloc

bps_laplace = sequential_universal_source_coding(compress_seq)

p_array = transformer_p_array(compress_seq.tolist(), model)
p_array = [0.5] + p_array[:-1]  # assume first bit is uniform (no prior)
bps_transformer = sequential_universal_source_coding(compress_seq, p_array=p

generated_p = convert_sample_to_pmf(compress_seq)
bps_entropy = entropy_calc(generated_p)

print(f'Entropy (lower bound): {bps_entropy:.4f}')
print(f'Laplace BPS:           {bps_laplace:.4f}')
print(f'Transformer BPS:       {bps_transformer:.4f}')
```

```
Entropy (lower bound): 0.6382
Laplace BPS:           0.6729
Transformer BPS:       0.6723
```

Note

Fix value of p instead of random in IID

Lets graph our stuff

In [67]:
```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

# Transformer vs Laplace prior quality at small N

def transformer_p_array_windowed_fast(sequence, model, device='cpu'):
    """
    Generate probability estimates for each position using a sliding window.
    For each position i (1-indexed), feeds the model the last min(i, block_s
    and uses the output logit at the last position to predict the next token
    """
    model.eval()
    B = model.block_size          # maximum context length the model can han
    T = len(sequence)
    p_array = [0.5]                # prior for the first bit (no context)

    # Move sequence to the correct device once
    seq_tensor = torch.tensor(sequence, dtype=torch.long, device=device)

    with torch.no_grad():
        for i in range(1, T):
            # Context = tokens before position i, limited to the last B toke
            start = max(0, i - B)
            context = seq_tensor[start:i].unsqueeze(0)  # shape [1, context_

            # Forward pass through the model
            logits, _ = model(context)                 # [1, context_len, vo
            # Take the logits from the last position (prediction for next to
            next_token_logits = logits[0, -1, :]       # [vocab_size]
            probs = F.softmax(next_token_logits, dim=-1)
            p_next = probs[1].item()                    # probability of tok
            p_array.append(p_next)

    return p_array

def run_experiment(model, p_values, N_values, n_trials=100, device='cpu'):
    """
    For each (p, N) pair, run n_trials with identical sequences for both met
    Returns nested dict: results[p][N] = {'laplace': (mean, std), 'transform
    """
    all_results = {}

    for p_true in p_values:
```

```
        print(f"\nRunning p={p_true}")
        p_dict = {0: 1 - p_true, 1: p_true}
        all_results[p_true] = {}

        for N in N_values:
            laplace_bps_trials = []
            transformer_bps_trials = []

            for trial in range(n_trials):
                # Same sequence for both methods
                seq = iid_generate(p_dict, N)

                # Laplace
                bps_lap = sequential_universal_source_coding(seq)
                laplace_bps_trials.append(bps_lap)

                # Transformer
                p_array = transformer_p_array_windowed_fast(seq, model, devi
                bps_trans = sequential_universal_source_coding(seq, p_array=
                transformer_bps_trials.append(bps_trans)

            all_results[p_true][N] = {
                'laplace': (np.mean(laplace_bps_trials), np.std(laplace_bps_
                'transformer': (np.mean(transformer_bps_trials), np.std(tran
                'entropy': -p_true * np.log2(p_true) - (1 - p_true) * np.log
            }
            print(f"  N={N}: laplace={np.mean(laplace_bps_trials):.4f}, tran

    return all_results
```

In [ ]:

```
# Run!!
p_values = [0.1, 0.3, 0.5, 0.7, 0.9]
N_values = [10, 25, 50, 100, 200, 500]

all_results = run_experiment(model, p_values, N_values, n_trials=100, device
```

In [69]:

```
def plot_bps_per_p(all_results, p_values, N_values):
    """One subplot per p value showing BPS convergence."""
    n_plots = len(p_values)
    fig, axes = plt.subplots(1, n_plots, figsize=(4 * n_plots, 4), dpi=150)
    if n_plots == 1:
        axes = [axes]

    for ax, p_true in zip(axes, p_values):
        means_lap = [all_results[p_true][N]['laplace'][0] for N in N_values]
        stds_lap  = [all_results[p_true][N]['laplace'][1] for N in N_values]
        means_tr  = [all_results[p_true][N]['transformer'][0] for N in N_val
        stds_tr   = [all_results[p_true][N]['transformer'][1] for N in N_val
        entropy   = all_results[p_true][N_values[0]]['entropy']

        ax.errorbar(N_values, means_lap, yerr=stds_lap, fmt='o-', capsize=4,
        ax.errorbar(N_values, means_tr,  yerr=stds_tr,  fmt='s-', capsize=4,
        ax.axhline(y=entropy, color='red', linestyle='--', label=f'Entropy (
        penalty = 0.5 * np.log2(np.array(N_values)) / np.array(N_values)
        ax.plot(N_values, entropy + penalty, 'g--', label='Entropy + USC pen
```

```
        ax.set_xscale('log')
        ax.set_title(f'p={p_true}')
        ax.set_xlabel('N')
        ax.set_ylabel('BPS')
        ax.legend(fontsize=7)
        ax.grid(True)

    plt.suptitle('Laplace vs Transformer BPS convergence', fontsize=12)
    plt.tight_layout()
    plt.show()


def plot_gap(all_results, p_values, N_values):
    """Plot the gap (laplace BPS - transformer BPS) per p. Positive = transf
    fig, ax = plt.subplots(figsize=(8, 4), dpi=150)

    for p_true in p_values:
        gaps = [
            all_results[p_true][N]['laplace'][0] - all_results[p_true][N]['t
            for N in N_values
        ]
        ax.plot(N_values, gaps, marker='o', label=f'p={p_true}')

    ax.axhline(y=0, color='black', linestyle='--', linewidth=1)
    ax.set_xscale('log')
    ax.set_xlabel('N')
    ax.set_ylabel('Laplace BPS - Transformer BPS')
    ax.set_title('Transformer advantage over Laplace (positive = transformer
    ax.legend()
    ax.grid(True)
    plt.tight_layout()
    plt.show()
```

In [2]:

```
import torch
from torch.utils.data import Dataset
from torch.utils.data.dataloader import DataLoader
from mingpt.utils import set_seed

class fixed_P(Dataset):
    """
    IID binary sequences
    """

    def __init__(self, n=10, num_samples=10000):
        self.n = n
        self.num_samples = num_samples

    def _sample(self):
        # sample probability p from uniform (0, 1)
        p_values = [0.1, 0.3, 0.5, 0.7, 0.9]
        p = np.random.choice(p_values)
        # generate IID Bernoulli sequence of length self.n
        seq = torch.bernoulli(torch.full((self.n,), p)).long()

        x = seq[:-1].clone()
        y = seq[1:].clone()
```

```
        return x, y

    def get_block_size(self):
        return self.n - 1

    def __getitem__(self, idx):
        return self._sample()

    def __len__(self):
        return self.num_samples

dataset_fixed_p = fixed_P(n=100, num_samples=10000)
```

In [3]:

```
# %%
# Define callback for training progress
def batch_end_callback(trainer):
    if trainer.iter_num % 100 == 0:
        print(f"iter_dt {trainer.iter_num}; iter {trainer.iter_num}: train l
```

In [ ]:

```
# %%
# Recreate model config for the fixed-p model
from mingpt.model import GPT
from mingpt.trainer import Trainer

model_config = GPT.get_default_config()
model_config.model_type = 'gpt-nano'
model_config.vocab_size = 2
model_config.block_size = dataset_fixed_p.get_block_size()
device = 'mps'
model_fixed_p = GPT(model_config)
model_fixed_p = model_fixed_p.to(device)

# %%
# Setup trainer for fixed-p model
train_config = Trainer.get_default_config()
train_config.learning_rate = 1e-4
train_config.max_iters = 10000
train_config.num_workers = 0
train_config.device = device


trainer_fixed_p = Trainer(train_config, model_fixed_p, dataset_fixed_p)

# Add callback
trainer_fixed_p.set_callback('on_batch_end', batch_end_callback)

# Train!
trainer_fixed_p.run()

# Save the trained model
torch.save(model_fixed_p.state_dict(), 'iid_model_fixed.pt')
print("Training complete! Model saved to 'iid_model_p07.pt'")
# Load with explicit device mapping
'''
```

```
'''
```

In [32]:
```
state_dict = torch.load('iid_model_p07.pt', map_location='cpu')  # Load to C
model_fixed_p.load_state_dict(state_dict)
model_fixed_p = model_fixed_p.to(device)
model_fixed_p.eval()
print(f'loaded! Model device: {next(model_fixed_p.parameters()).device}')
```

```
loaded! Model device: mps:0
```

In [60]:
```
p_values = [0.7]
N_values = [10, 25, 50, 100, 200, 500]


p7_experiment_results = run_experiment(model_fixed_p, p_values, N_values, n_
```

```
Running p=0.7
  N=10: laplace=0.9719, transformer=0.8870
  N=25: laplace=0.9449, transformer=0.8895
  N=50: laplace=0.9164, transformer=0.8772
  N=100: laplace=0.9061, transformer=0.8829
  N=200: laplace=0.8956, transformer=0.8823
  N=500: laplace=0.8903, transformer=0.8837
```

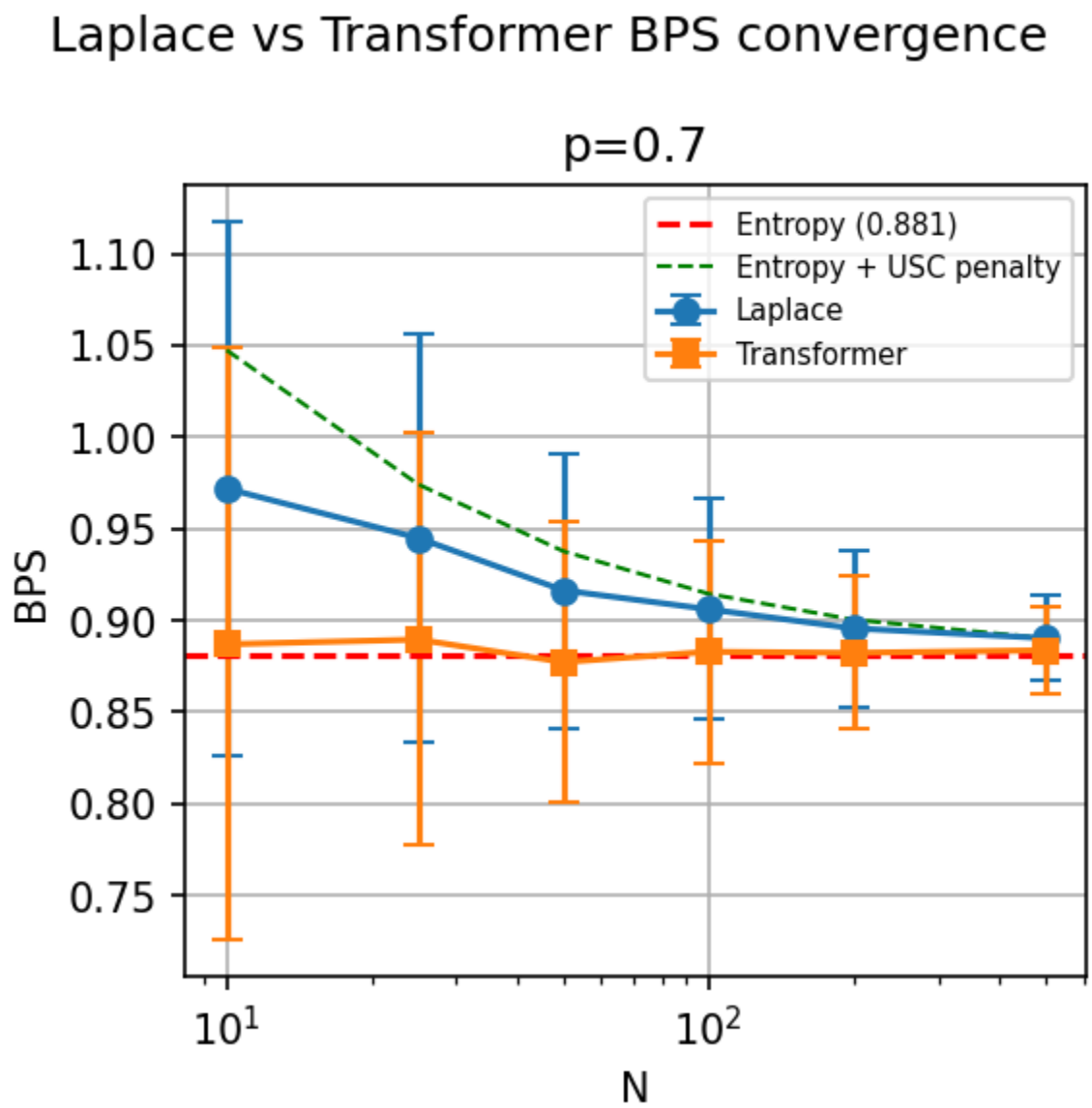
In [73]:
```
#save shit
import pickle

with open('p7_results_fixed.pkl', 'wb') as f:
    pickle.dump(p7_experiment_results, f)
```

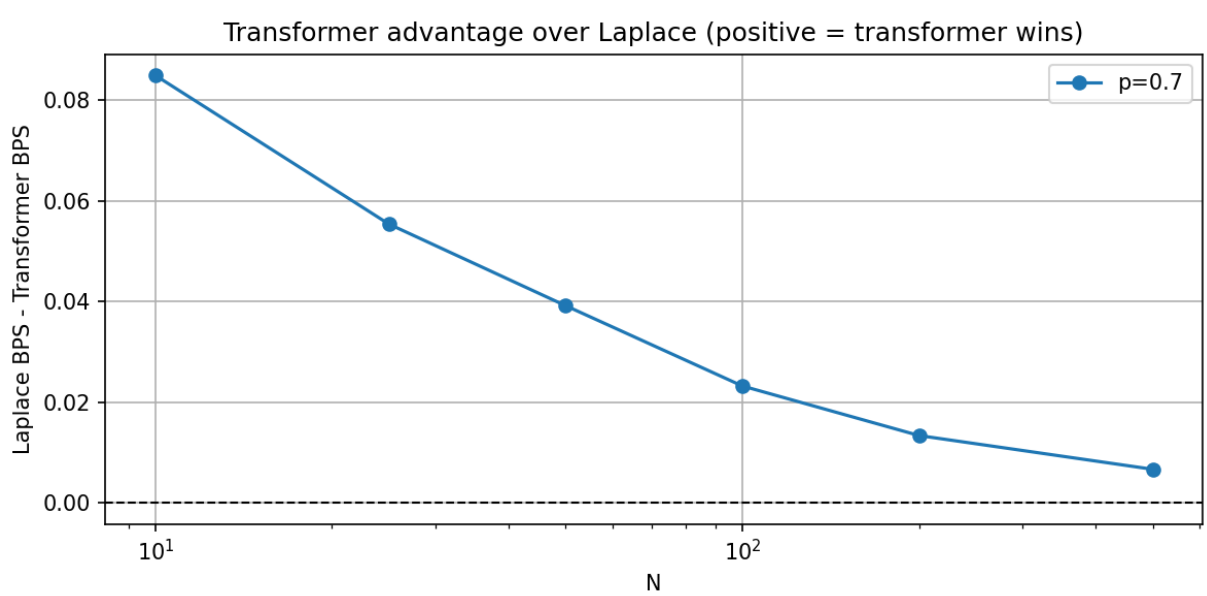
In [74]:
```
with open('p7_results_fixed.pkl', 'rb') as f:
    p7_results_fixed = pickle.load(f)
print(p7_results_fixed)
```

```
{0.7: {10: {'laplace': (np.float64(0.9718765576703483), np.float64(0.1459842
2480312262)), 'transformer': (np.float64(0.8869818367888703), np.float64(0.1
6134283259654067)), 'entropy': np.float64(0.8812908992306927)}, 25: {'laplac
e': (np.float64(0.9449020721004062), np.float64(0.11169789292807485)), 'tran
sformer': (np.float64(0.8895483958058098), np.float64(0.1123907652543899)),
'entropy': np.float64(0.8812908992306927)}, 50: {'laplace': (np.float64(0.91
63807327513684), np.float64(0.07489817846188696)), 'transformer': (np.float6
4(0.8772328368213297), np.float64(0.07619637900212956)), 'entropy': np.float
64(0.8812908992306927)}, 100: {'laplace': (np.float64(0.9060822474650659), n
p.float64(0.06015932260434652)), 'transformer': (np.float64(0.88287731188461
9), np.float64(0.06064020876406023)), 'entropy': np.float64(0.88129089923069
27)}, 200: {'laplace': (np.float64(0.8956453493142864), np.float64(0.0426470
68198004165)), 'transformer': (np.float64(0.8822872264714324), np.float64
(0.04179980675293816)), 'entropy': np.float64(0.8812908992306927)}, 500: {'l
aplace': (np.float64(0.890328228762623), np.float64(0.023056744977237267)),
'transformer': (np.float64(0.8836681939536614), np.float64(0.023629012494709
924)), 'entropy': np.float64(0.8812908992306927)}}}
```

In [77]:
```
p_values = [0.7]
N_values = [10, 25, 50, 100, 200, 500]
plot_bps_per_p(p7_results_fixed, p_values, N_values)
```

```
plot_gap(p7_results_fixed, p_values, N_values)
```





In [ ]:
```
p_values = [0.1, 0.3, 0.5, 0.7, 0.9]
N_values = [10, 25, 50, 100, 200, 500]


p_experiment_results = run_experiment(model_fixed_p, p_values, N_values, n_t
```

In [ ]:
```
#save shit
import pickle

with open('p_results_fixed.pkl', 'wb') as f:
    pickle.dump(p_experiment_results, f)
```

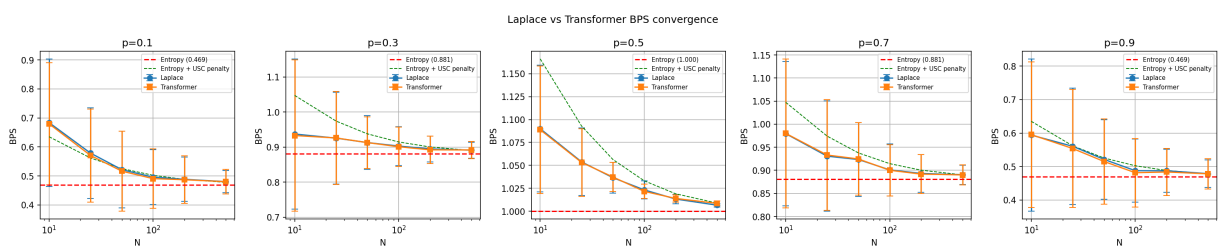
In [61]:
```
import pickle

with open('p_results_fixed.pkl', 'rb') as f:
    p_results_fixed = pickle.load(f)
print(p_results_fixed)
```

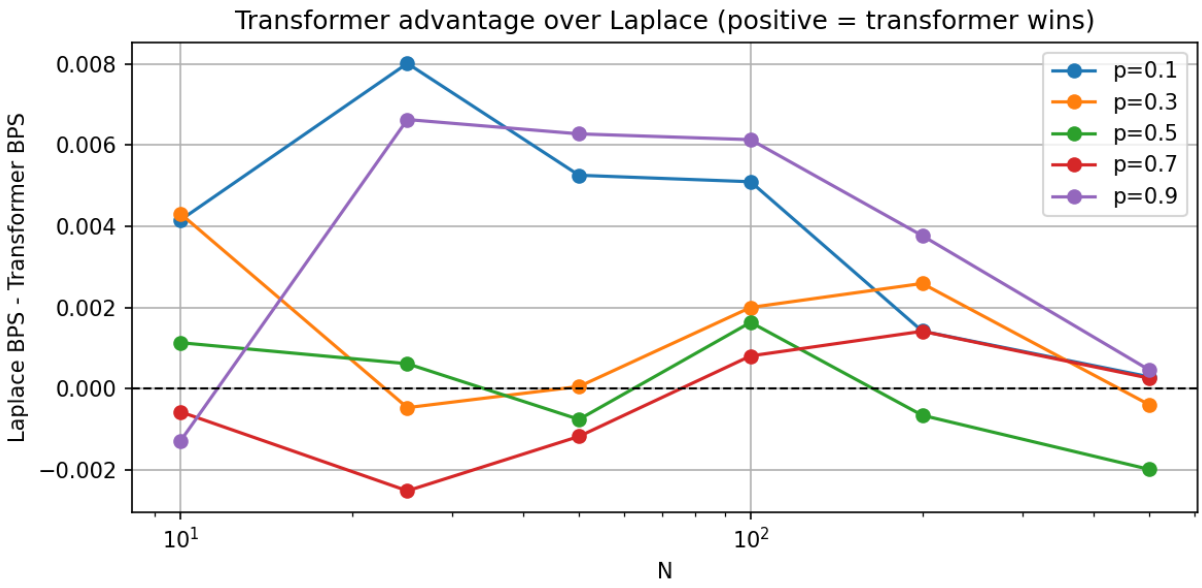
{0.1: {10: {'laplace': (np.float64(0.683759586883169), np.float64(0.21974368
848614279)), 'transformer': (np.float64(0.6796054538286346), np.float64(0.21
066197384802446)), 'entropy': np.float64(0.4689955935892812)}, 25: {'laplac
e': (np.float64(0.5786884399224418), np.float64(0.15587069856654676)), 'tran
sformer': (np.float64(0.570680414519325), np.float64(0.1604253058892503)), '
entropy': np.float64(0.4689955935892812)}, 50: {'laplace': (np.float64(0.521
732159394033), np.float64(0.1323137841956996)), 'transformer': (np.float64
(0.5164758436534888), np.float64(0.13831087479073023)), 'entropy': np.float6
4(0.4689955935892812)}, 100: {'laplace': (np.float64(0.49584686824056795), n
p.float64(0.09511479796374696)), 'transformer': (np.float64(0.49075028670797
7), np.float64(0.10262752718610214)), 'entropy': np.float64(0.46899559358928
12)}, 200: {'laplace': (np.float64(0.4883508814201319), np.float64(0.0763177
8412720666)), 'transformer': (np.float64(0.486928929238099), np.float64(0.08
213863923948625)), 'entropy': np.float64(0.4689955935892812)}, 500: {'laplac
e': (np.float64(0.4804562771226714), np.float64(0.037993822714236986)), 'tra
nsformer': (np.float64(0.4801569833014934), np.float64(0.04238757426007172
5)), 'entropy': np.float64(0.4689955935892812)}}, 0.3: {10: {'laplace': (n
p.float64(0.9372123955701734), np.float64(0.2143231711600683)), 'transformer
': (np.float64(0.9329012938102285), np.float64(0.21602370193852438)), 'entro
py': np.float64(0.8812908992306927)}, 25: {'laplace': (np.float64(0.92550806
43265741), np.float64(0.13126223032992637)), 'transformer': (np.float64(0.92
5969170271447), np.float64(0.13253814002579256)), 'entropy': np.float64(0.88
12908992306927)}, 50: {'laplace': (np.float64(0.9128043313281609), np.float6
4(0.07666006555796316)), 'transformer': (np.float64(0.912746205775435), np.f
loat64(0.07387113649311869)), 'entropy': np.float64(0.8812908992306927)}, 10
0: {'laplace': (np.float64(0.9029447385691194), np.float64(0.055500884191758
124)), 'transformer': (np.float64(0.9009404801378369), np.float64(0.05583610
1106496824)), 'entropy': np.float64(0.8812908992306927)}, 200: {'laplace':
(np.float64(0.8947578899772464), np.float64(0.036721287497920606)), 'transfo
rmer': (np.float64(0.8921639103933089), np.float64(0.03914901299562795)), 'e
ntropy': np.float64(0.8812908992306927)}, 500: {'laplace': (np.float64(0.891
3598191689508), np.float64(0.02288073910888727)), 'transformer': (np.float64
(0.8917461893730447), np.float64(0.02466648708104756)), 'entropy': np.float6
4(0.8812908992306927)}}, 0.5: {10: {'laplace': (np.float64(1.089999084259786
9), np.float64(0.06902702803607458)), 'transformer': (np.float64(1.088862087
0738317), np.float64(0.0693519963083667)), 'entropy': np.float64(1.0)}, 25:
{'laplace': (np.float64(1.0538596929491717), np.float64(0.0368455660160836
3)), 'transformer': (np.float64(1.0532399511843074), np.float64(0.0368168586
6163897)), 'entropy': np.float64(1.0)}, 50: {'laplace': (np.float64(1.036621
6021326489), np.float64(0.01683076898339578)), 'transformer': (np.float64
(1.0373721049832603), np.float64(0.015716628049545336)), 'entropy': np.float
64(1.0)}, 100: {'laplace': (np.float64(1.0233465368179848), np.float64(0.009
775625064872017)), 'transformer': (np.float64(1.0217088368124074), np.float6
4(0.007849713734622888)), 'entropy': np.float64(1.0)}, 200: {'laplace': (n
p.float64(1.0133624961770074), np.float64(0.004983295657981709)), 'transform
er': (np.float64(1.0140113418278427), np.float64(0.003885040050639241)), 'en
tropy': np.float64(1.0)}, 500: {'laplace': (np.float64(1.00667384829242), n
p.float64(0.0021055738707170643)), 'transformer': (np.float64(1.008652464084
3606), np.float64(0.0028414145736441712)), 'entropy': np.float64(1.0)}}, 0.7
: {10: {'laplace': (np.float64(0.9795325104966957), np.float64(0.15642462230
754414)), 'transformer': (np.float64(0.9800963910955428), np.float64(0.16140
247552219236)), 'entropy': np.float64(0.8812908992306927)}, 25: {'laplace':
(np.float64(0.9307277472372946), np.float64(0.11939170709751891)), 'transfor
mer': (np.float64(0.9332347815939732), np.float64(0.11982355881154108)), 'en
tropy': np.float64(0.8812908992306927)}, 50: {'laplace': (np.float64(0.92345
3589686025), np.float64(0.07982261142104942)), 'transformer': (np.float64

```
(0.9246223340226264), np.float64(0.0788173394931304)), 'entropy': np.float64
(0.8812908992306927)}, 100: {'laplace': (np.float64(0.9009769953726628), n
p.float64(0.05640788515699092)), 'transformer': (np.float64(0.90016547585362
84), np.float64(0.05542517140891617)), 'entropy': np.float64(0.8812908992306
927)}, 200: {'laplace': (np.float64(0.8933258545312639), np.float64(0.041270
49298369202)), 'transformer': (np.float64(0.8919077863008197), np.float64
(0.04213425779296)), 'entropy': np.float64(0.8812908992306927)}, 500: {'lapl
ace': (np.float64(0.8904488179977109), np.float64(0.02081570256131417)), 'tr
ansformer': (np.float64(0.8901847333949356), np.float64(0.02207377725731768
7)), 'entropy': np.float64(0.8812908992306927)}}, 0.9: {10: {'laplace': (n
p.float64(0.5939728494115457), np.float64(0.22792613572437684)), 'transforme
r': (np.float64(0.5952546853988079), np.float64(0.2178259604687089)), 'entro
py': np.float64(0.4689955935892811)}, 25: {'laplace': (np.float64(0.56045131
21035289), np.float64(0.17417385441435457)), 'transformer': (np.float64(0.55
3824895571961), np.float64(0.17683853619240428)), 'entropy': np.float64(0.46
89955935892811)}, 50: {'laplace': (np.float64(0.5213214495663955), np.float6
4(0.11910554515434456)), 'transformer': (np.float64(0.5150476589144762), n
p.float64(0.12768073660173063)), 'entropy': np.float64(0.4689955935892811)},
100: {'laplace': (np.float64(0.48751127887086876), np.float64(0.094521671774
99247)), 'transformer': (np.float64(0.48137987465045234), np.float64(0.10257
041016120734)), 'entropy': np.float64(0.4689955935892811)}, 200: {'laplace':
(np.float64(0.4873331492335702), np.float64(0.06420094374799631)), 'transfor
mer': (np.float64(0.4835628979443504), np.float64(0.07034121112259352)), 'en
tropy': np.float64(0.4689955935892811)}, 500: {'laplace': (np.float64(0.4785
9905446949896), np.float64(0.04165184025433750 5)), 'transformer': (np.float6
4(0.4781309525464102), np.float64(0.04566344673743783)), 'entropy': np.float
64(0.4689955935892811)}}}
```

In [ ]:

```
p_values = [0.1, 0.3, 0.5, 0.7, 0.9]
N_values = [10, 25, 50, 100, 200, 500]
plot_bps_per_p(p_results_fixed, p_values, N_values)
plot_gap(p_results_fixed, p_values, N_values)
```




Transformer advantage over Laplace (positive = transformer wins)
Laplace BPS - Transformer BPS
0.008
0.006
0.004
0.002
0.000
−0.002
10^1
10^2
N
p=0.1
p=0.3
p=0.5
p=0.7
p=0.9